

Nama: Naja Dina Fatiha
NIM: 245150600111010
Kelas: PTI A - PAPB

4.5.1 State Hosting

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.

a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Jawab: Setiap kali tombol **Increment** diklik, nilai number bertambah 1. Misalnya nilai awal 0, setelah satu kali klik menjadi 1, setelah dua kali klik menjadi 2, dan seterusnya.

b. Apa yang menyebabkan hal tersebut terjadi?

Jawab: Hal tersebut terjadi karena pada Button terdapat kode: `onClick = { number++ }`. Setiap tombol diklik, nilai state number bertambah satu. Perubahan state tersebut menyebabkan Compose melakukan *recomposition* sehingga nilai terbaru ditampilkan pada Text.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Jawab: Variabel state yang berperan adalah: number karena nilai tersebut digunakan oleh: `Text(text = "$number")`

d. Variabel state apa yang berperan untuk menampilkan label teks pada

Jawab: Variabel state yang digunakan adalah: label yang digunakan pada: `Text(text = "$label")`

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Jawab: Perubahan dilakukan pada fungsi CounterScreen().

Dari: `number++`

Menjadi: `number += 2`

Kemudian label: "Increment"

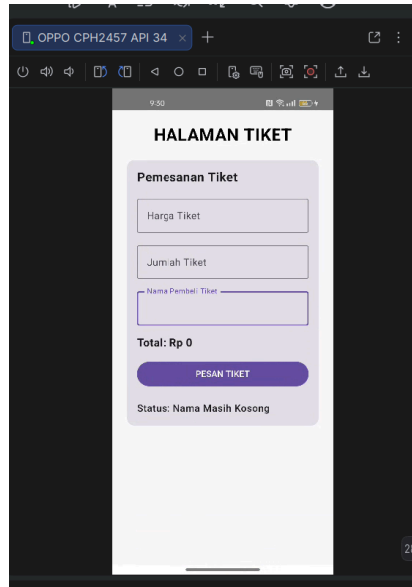
diubah menjadi: "Increment 2"

Jadi setiap tombol diklik, angka bertambah 2.

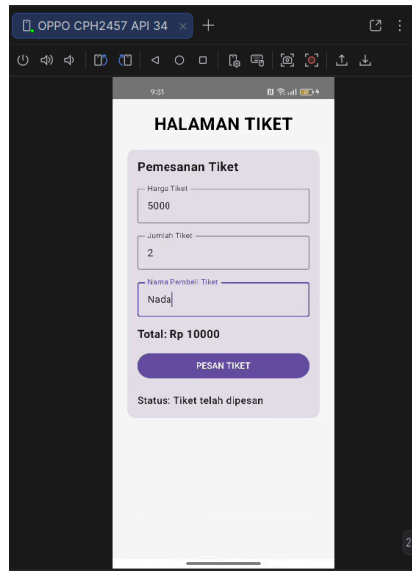
b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Jawab:

Sebelum:



Sesudah:



3. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Jawab: Ya. Program tetap berjalan dengan baik dan menghasilkan tampilan serta fungsi yang sama. Perbedaannya adalah lokasi pengelolaan state dan event sudah dipindahkan ke composable induk.

b. Di bagian kode program manakah saat ini proses increment dilakukan?

Jawab: Proses increment dilakukan pada lambda:

```
onButtonClick = {
    number++
```

} yang berada di Parent, bukan lagi di dalam CounterScreen.

c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

Jawab: State number dideklarasikan pada Parent:

```
var number by remember {  
    mutableStateOf(0)  
}
```

Sedangkan CounterScreen hanya menerima: number yaitu Int dan label: String sebagai parameter.

4. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

a. Label tombol "Increment" menjadi "Double"?

Jawab: Double

b. Nilai awal variabel number dari 0 menjadi 1.

Jawab: 1

c. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.

Jawab: dari number++ menjadi number *=2 atau number = number * 2

d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
Jawab: var number by remember {  
    mutableStateOf(1)  
}  
CounterScreen(  
    Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = {  
        number *= 2  
    }  
)
```

5. b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Jawab: Perubahan dilakukan pada composable Parent, khususnya bagian:

```
var number by remember {  
    mutableStateOf(1)  
}
```

dan:

```
onButtonClick = {  
    number *= 2  
}
```

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Jawab: Untuk perubahan fungsi dari Increment menjadi Double, **tidak ada perubahan logika perhitungan di dalam CounterScreen**. CounterScreen hanya menampilkan nilai dan menjalankan callback yang diberikan Parent. Perubahan logika dilakukan pada Parent.

6. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Jawab: State Hoisting membuat perubahan kebutuhan fungsional lebih mudah dilakukan karena logika state dan event berada di Parent. Ketika kebutuhan berubah dari menambah angka menjadi menggandakan angka, cukup mengubah lambda pada Parent tanpa mengubah logika utama Composable anak. Dengan demikian Composable anak menjadi lebih *reusable*, lebih mudah diuji, dan memiliki tanggung jawab yang lebih sederhana. Hal tersebut sesuai dengan tujuan State Hoisting pada modul, yaitu membuat komponen lebih mudah digunakan kembali, diuji, serta memiliki *Single Source of Truth*.

4.5.2 Perubahan Konfigurasi

2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawab: Jika nilai awal: 1 kemudian tombol Double ditekan beberapa kali, misalnya:

1 → 2 → 4 → 8 maka setelah rotasi dengan remember, nilai dapat kembali menjadi: 1

Hal tersebut terjadi karena remember tidak mempertahankan state ketika Activity dibuat kembali akibat perubahan konfigurasi.

3. b. Setelah menggunakan rememberSaveable, berapa angka yang ditampilkan?

Jawab: Angka tetap sama seperti sebelum rotasi.

Contohnya: 1 → 2 → 4 → 8

Setelah layar diputar: 8

tetap ditampilkan.

c. Apa fungsi dari konstruk rememberSaveable pada kode program Android yang membedakannya dari konstruk remember?

Jawab: rememberSaveable digunakan untuk mempertahankan nilai state ketika terjadi perubahan konfigurasi seperti rotasi layar. remember mempertahankan state selama *recomposition*, sedangkan rememberSaveable juga menggunakan mekanisme saved instance state sehingga nilai yang dapat disimpan tetap tersedia ketika Activity dibuat kembali.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Jawab: Contoh yang tepat adalah halaman formulir pendaftaran. Misalnya pengguna sudah mengisi nama, alamat, nomor telepon, dan email, kemudian perangkat diputar ke posisi landscape. Dengan `rememberSaveable`, data yang sudah dimasukkan dapat dipertahankan sehingga pengguna tidak perlu mengetik ulang seluruh data.

4.5.3 Launched

1. Jalankan program,

c. Pada saat pertama kali program dijalankan, tanpa mengklik button Double apa yang ditampilkan di dalam layar?

Jawab: Pada percobaan modul, nilai awal number adalah 1 kemudian Snackbar menampilkan number 1 is shown!

d. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Jawab: Pesan muncul tanpa interaksi pengguna Karena: `LaunchedEffect(number)` dijalankan ketika Composable pertama kali masuk ke komposisi. Jadi pengguna tidak perlu menekan tombol terlebih dahulu. Modul menjelaskan bahwa `LaunchedEffect` dapat menjalankan blok kode ketika Composable pertama kali masuk ke komposisi.

e. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Jawab: Yang memicu adalah `LaunchedEffect` dengan key: number Kode yang digunakan modul:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar(  
        "Number $number is shown!"  
    )  
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

Jawab: Nilai number menjadi dua kali lipat. Misalnya:

1 → 2 Kemudian karena nilai number berubah, `LaunchedEffect(number)` dijalankan kembali sehingga Snackbar menampilkan nilai terbaru.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

Jawab: Karena number digunakan sebagai key dari `LaunchedEffect`.

Ketika:

number = 1

berubah menjadi:

number = 2

key berubah sehingga efek lama dibatalkan dan efek dengan nilai key baru dijalankan kembali.

3. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

Jawab: Efek dijalankan ketika `Composable` pertama kali masuk komposisi dan dijalankan kembali ketika nilai state yang menjadi key berubah.

Contoh:

```
LaunchedEffect(number)
```

Jika:

`number = 1`

kemudian:

`number = 2`

maka efek dijalankan kembali.

`LaunchedEffect(Unit)` Digunakan untuk efek yang ingin dijalankan ketika `Composable` masuk ke komposisi dan tidak bergantung pada perubahan state tertentu sebagai key.

Contoh:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` Dijalankan?

Jawab: `LaunchedEffect(state)` dijalankan ketika `Composable` masuk ke komposisi dan ketika nilai state sebagai key berubah.

Sedangkan:

`LaunchedEffect(Unit)` dijalankan ketika `Composable` masuk ke komposisi. Selama key `Unit` tidak berubah, efek tersebut tidak dijalankan ulang hanya karena terjadi *recomposition*.

Tugas Praktikum Buat 1 halaman dengan Tema Halaman Tiket

Terapkan `StateHoisting` dengan State yang dikelola Parent:

- 1 . Harga Tiket
- 2 . Jumlah Tiket
3. Nama Pembeli Tiket

Gunakan `Launched Effect` untuk menampilkan :

Status : Nama Masih Kosong

Status : Memproses pesanan....., setelah 5 detik, muncul lagi

Status : Tiket telah dipesan

Jawab:

Link GitHub: <https://github.com/nadastala/TiketApp.git>

1. Kode `MainActivity.kt`

```
package com.example.tiketapp
```

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlinx.coroutines.delay

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MaterialTheme {
                TicketScreen()
            }
        }
    }
}

/*
 * PARENT
 * Semua state utama dikelola di sini.
 */
@Composable

fun TicketScreen() {

```

```

var hargaTiket by remember {
    mutableStateOf("50000")
}

var jumlahTiket by remember {
    mutableStateOf("1")
}

var namaPembeli by remember {
    mutableStateOf("")
}

var status by remember {
    mutableStateOf("Nama Masih Kosong")
}

var orderTrigger by remember {
    mutableStateOf(0)
}

/*
 * LaunchedEffect berjalan ketika orderTrigger
berubah.
 */
LaunchedEffect(orderTrigger) {

    if (orderTrigger > 0) {

        status = "Memproses pesanan....."

        delay(5000)

        status = "Tiket telah dipesan"
    }
}

TicketPage(
    hargaTiket = hargaTiket,
    jumlahTiket = jumlahTiket,
    namaPembeli = namaPembeli,
    status = status,

    onHargaChange = {
        hargaTiket = it
    },

    onJumlahChange = {
        jumlahTiket = it
    },

    onNamaChange = {

```

```

        namaPembeli = it
    },

    onPesanClick = {

        if (namaPembeli.isBlank()) {

            status = "Nama Masih Kosong"

        } else {

            orderTrigger++

        }

    }

)

}

/*
 * CHILD
 * Tidak menyimpan state harga, jumlah, atau nama.
 * Semua data diberikan oleh Parent melalui parameter.
 */
@Composable
fun TicketPage(
    hargaTiket: String,
    jumlahTiket: String,
    namaPembeli: String,
    status: String,

    onHargaChange: (String) -> Unit,
    onJumlahChange: (String) -> Unit,
    onNamaChange: (String) -> Unit,
    onPesanClick: () -> Unit
) {

    val harga = hargaTiket.toLongOrNull() ?: 0L
    val jumlah = jumlahTiket.toLongOrNull() ?: 0L
    val total = harga * jumlah

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Top,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {

        Text(
            text = "HALAMAN TIKET",
            fontSize = 28.sp,

```

```

        fontWeight = FontWeight.Bold
    )

    Spacer(modifier = Modifier.height(24.dp))

    Card(
        modifier = Modifier.fillMaxWidth()
    ) {

        Column(
            modifier = Modifier.padding(16.dp)
        ) {

            Text(
                text = "Pemesanan Tiket",
                fontSize = 20.sp,
                fontWeight = FontWeight.Bold
            )

            Spacer(modifier = Modifier.height(16.dp))

            OutlinedTextField(
                value = hargaTiket,
                onValueChange = onHargaChange,
                label = {
                    Text("Harga Tiket")
                },
                modifier = Modifier.fillMaxWidth(),
                keyboardOptions = KeyboardOptions(
                    keyboardType = KeyboardType.Number
                )
            )

            Spacer(modifier = Modifier.height(12.dp))

            OutlinedTextField(
                value = jumlahTiket,
                onValueChange = onJumlahChange,
                label = {
                    Text("Jumlah Tiket")
                },
                modifier = Modifier.fillMaxWidth(),
                keyboardOptions = KeyboardOptions(
                    keyboardType = KeyboardType.Number
                )
            )

            Spacer(modifier = Modifier.height(12.dp))

            OutlinedTextField(
                value = namaPembeli,

```

```

        onValueChange = onNamaChange,
        label = {
            Text("Nama Pembeli Tiket")
        },
        modifier = Modifier.fillMaxWidth()
    )

    Spacer(modifier = Modifier.height(16.dp))

    Text(
        text = "Total: Rp $total",
        fontSize = 18.sp,
        fontWeight = FontWeight.Bold
    )

    Spacer(modifier = Modifier.height(16.dp))

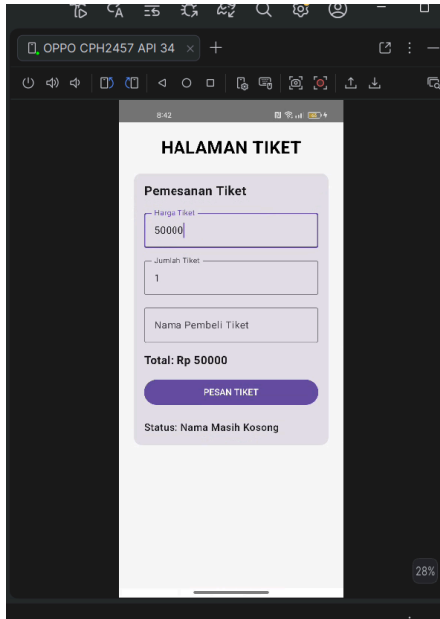
    Button(
        onClick = onPesanClick,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("PESAN TIKET")
    }

    Spacer(modifier = Modifier.height(20.dp))

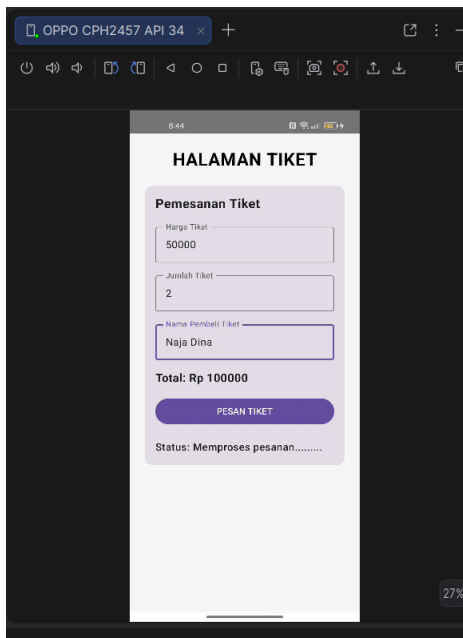
    Text(
        text = "Status: $status",
        fontSize = 16.sp,
        fontWeight = FontWeight.Medium
    )
}
}
}
}
}

```

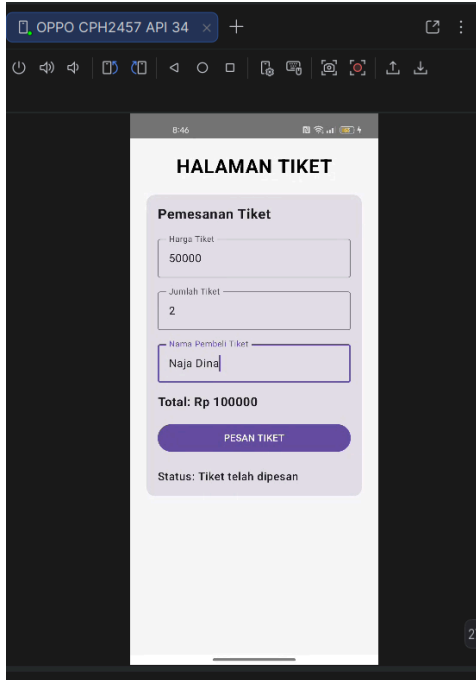
2. Screenshot kondisi awal



3. Screenshot memproses pesanan



4. Screenshot Tiket telah dipesan



5. Screenshot Nama Kosong

